MCPify

> Compile software into AI-operable systems.

MCPify is an AI enablement compiler that transforms existing applications into AI-native, agent-operable systems.

Instead of manually building MCP servers, AI interfaces, workflows, and integrations, MCPify automatically analyzes backend code, frontend applications, APIs, databases, workflows, and permissions to generate secure AI-operable abstractions.

The goal is simple:

> Any software should become usable by AI agents instantly.

The Vision

Software today is designed for humans.

Buttons, APIs, forms, workflows, dashboards, and databases are built around human interaction.

AI agents are rapidly becoming operational systems capable of:

- coding
- browsing
- planning
- executing tasks
- interacting with tools
- operating software

However, modern applications are not designed for AI-native interaction.

Developers currently need to manually:

- expose APIs
- write MCP servers
- define tools
- create schemas
- implement safety layers
- maintain synchronization

This creates massive friction.

MCPify solves this by acting as an:

AI Enablement Compiler

It converts software systems into structured, secure, AI-operable environments automatically.

Core Concept

Traditional software stack:

```txt id="2c0dja" Frontend Backend Database APIs Workflows Permissions
```

```
MCPify transforms this into:

```txt id="92imvr"
AI Tools
AI Workflows
AI Actions
Semantic Interfaces
Permission-Aware Operations
Agent-Compatible Systems
```

The Core Idea

```bash id="4v4zy8" npx mcpify
```

```
MCPify scans:

- frontend
- backend
- APIs
- database models
- workflows
- events
- permissions

and generates:

- MCP servers
- AI tools
```
```

```
- semantic workflows
- safety boundaries
- action abstractions
- AI-operable interfaces

---

# What Makes MCPify Different

Most MCP tools only expose backend functions.

MCPify understands:

- applications
- workflows
- actions
- intent
- permissions
- UI interactions
- system structure

It is not merely:
> “Function wrapping”

It is:
> “Application understanding for AI systems.”

---

# Problem Statement

Current AI integrations face several problems.

---

# 1. Manual MCP Boilerplate

Developers repeatedly write:

- tool schemas
- validation
- wrappers
- metadata
- handlers

for systems that already exist.

---
```

```
# 2. Frontends Are Invisible to AI

AI agents struggle to interact with:

- UI workflows
- buttons
- forms
- modals
- state changes

without browser automation hacks.

---

# 3. Applications Lack Semantic Structure

Existing systems are not represented in a way AI agents can reason about.

AI sees:

- raw APIs
- raw buttons
- raw endpoints

instead of:

- workflows
- intent
- actions
- safe operations

---

# 4. Security Risks

Exposing software to AI systems without:

- permissions
- safety constraints
- operation boundaries

creates dangerous systems.

---

# 5. Synchronization Drift
```

````
As code changes:

- MCP definitions become outdated
- docs drift
- tools desynchronize

---

# Solution

MCPify automatically:

- analyzes application structure
- extracts semantic actions
- generates AI tools
- builds workflows
- creates permission-aware interfaces
- synchronizes changes continuously

---

# High-Level Architecture

```txt id="agmg7m"
Application
   ↓
Static Analysis Engine
   ↓
Semantic Understanding Layer
   ↓
Workflow Extraction
   ↓
Safety & Permission Layer
   ↓
MCP Generation
   ↓
AI-Operable System
````

````

System Components

1. Backend Analyzer

Scans:

- TypeScript
- JavaScript
- APIs
- services
- controllers
- SDKs

Extracts:

- functions
- types
- actions
- workflows
- dependencies

Example

Input:

```ts id="rjvcfv" export async function refundOrder(orderId: string) {}
```

Generated:

```txt id="0gux31"
refundOrder(orderId)
```

with:

- schema
- validation
- permissions
- AI metadata

---

2. Frontend Analyzer

One of MCPify's most important features.

Scans:

- React
- Next.js
- Vue
- Angular

Extracts:

- buttons
- forms
- routes
- state mutations
- UI actions
- workflows

Example

Input:

```tsx id="gux4ol"
<button onClick={checkout}>
  Checkout
</button>
```

```
Generated AI Action:

```txt id="v6jlwm"
checkoutCart()
```

This allows AI systems to operate applications directly.

Semantic Frontend Understanding

MCPify does not only detect UI elements.

It understands intent.

Example:
```

```txt id="0z20sg" "Submit Support Ticket"

```
becomes:

```txt id="w88nvc"
createSupportRequest()
```

This creates structured AI-operable interfaces instead of fragile browser automation.

---

3. Workflow Extraction Engine

MCPify automatically identifies workflows.

Example:

```txt id="a8h6sl" Login → Add Product → Checkout → Payment

```
Generated:

```txt id="2w2pxn"
purchaseWorkflow()
```

This enables:

- autonomous execution
- multi-step operations
- reusable AI workflows

---

4. Database Analyzer

Scans:

- Prisma
- Drizzle
- Mongoose
- SQLAlchemy
- PostgreSQL schemas

Generates:

- query tools
- filtered access tools
- semantic data operations

Example

Input:

```prisma id="7g6s0o" model Order { id String status String }

```
Generated:

```txt id="sb4eqp"
getOrdersByStatus()
```

---

5. API → MCP Conversion

Convert APIs instantly into AI-native tools.

Example:

```bash id="7p0lvr" npx mcpify swagger.json

```
Generated:

- endpoints
- schemas
- validations
- tools
- workflows

This enables immediate AI integration for:

- SaaS platforms
- internal systems
- microservices

```
```

````
# 6. Event System Integration

Scans:

- Kafka
- RabbitMQ
- Webhooks
- EventEmitter
- Pub/Sub systems

Generates:

- event listeners
- subscriptions
- reactive workflows

This allows AI agents to become event-aware systems.

---

# 7. Permission & Safety Layer

Critical for production AI systems.

Every generated tool receives:

- permission levels
- safety classifications
- approval requirements
- access boundaries

---

# Example

```txt id="s18db6"
SAFE:
viewOrders()

REQUIRES_CONFIRMATION:
refundOrder()

BLOCKED:
deleteDatabase()
````
````

8. AI Metadata Enhancement

Optional AI enhancement layer.

Improves:

- descriptions
- examples
- naming
- parameter clarity
- workflow understanding

Example

Input:

```ts id="fq2q8x" function f(q){}
```

Enhanced:

```txt id="r9o0q6"
Searches internal documentation using a text query.
```

---

9. AGENTS.md Generation

Automatically generates:

- architecture overview
- workflows
- commands
- repository structure
- operational guidelines

Useful for:

- AI coding agents
- onboarding
- autonomous systems

---

10. Self-Updating Synchronization

MCPify continuously synchronizes generated AI layers with source code.

When code changes:

- schemas update
- workflows regenerate
- permissions refresh
- MCP definitions stay synchronized

This prevents:

- MCP drift
- stale tools
- invalid schemas

---

11. AI Simulation & Validation

MCPify can simulate AI usage.

Tests:

- prompt injection
- invalid operations
- workflow failures
- permission bypass attempts
- unsafe operations

Example

```bash id="ul9cxp" npx mcpify simulate
```

```
---

# 12. Knowledge Graph Engine

Advanced future feature.

Builds semantic graph of:
```

```
- UI
- APIs
- workflows
- permissions
- services
- data relationships

Allows AI systems to reason about entire applications contextually.

---

# Core Philosophy

MCPify is designed around one idea:

> Applications should become AI-operable automatically.

The goal is not simply exposing APIs.

The goal is creating:

- structured AI interfaces
- semantic operations
- safe execution environments
- workflow-aware systems

---

# Example End-to-End Flow

# Existing SaaS Application

Contains:

- frontend dashboard
- backend API
- PostgreSQL database
- checkout workflow

---

# Run MCPify

```bash id="9d5yzf"
npx mcpify
```
```

Generated Output

```txt id="hkmhqq" ✓ createOrder() ✓ checkoutCart() ✓ refundOrder() ✓ purchaseWorkflow() ✓ getOrdersByStatus() ✓ createSupportTicket()

```

Connect to AI Agent

Use:

- Codex
- Claude Desktop
- Cursor
- autonomous agents

Example AI Interaction

User:

```txt id="tbrv2y"
Refund the latest failed order and notify the customer.
```

AI:

- identifies workflow
- checks permissions
- executes refund
- sends notification
- updates state

without manually written MCP tooling.

Developer Experience

One Command

```bash id="cftv9n" npx mcpify
```

````
---

# Interactive Mode

```bash id="yb0j3g"
npx mcpify --interactive
````

Audit Mode

```bash id="tctn4r" npx mcpify --audit

````

Frontend Extraction

```bash id="ye1xhz"
npx mcpify frontend
````

OpenAPI Conversion

```bash id="vrgdfj" npx mcpify swagger.json

````

Suggested Tech Stack

Area	Technology
CLI	commander.js
AST Parsing	ts-morph
Validation	zod
Frontend Parsing	Babel / SWC
MCP Integration	MCP SDK
Formatting	prettier
OpenAPI Parsing	swagger-parser
Knowledge Graph	Neo4j / graphlib
AI Enhancement	OpenAI API

````

```
# Proposed Repository Structure

```txt id="mjlwm4"
mcpify/
├── packages/
│   ├── cli/
│   ├── backend-analyzer/
│   ├── frontend-analyzer/
│   ├── workflow-engine/
│   ├── schema-engine/
│   ├── mcp-generator/
│   ├── permissions/
│   ├── security/
│   ├── ai-enhancer/
│   ├── graph-engine/
│   └── templates/
```

Roadmap

Phase 1 — Hackathon MVP

- backend analysis
- schema generation
- MCP generation
- OpenAPI support
- frontend action extraction

Phase 2

- workflow understanding
- semantic UI understanding
- permissions
- audit system

Phase 3

- knowledge graph
- event systems
```

- synchronization engine
- AI simulations

Phase 4

- enterprise deployment
- hosted platform
- monitoring
- analytics
- AI operating layer

Long-Term Vision

MCPify aims to become the infrastructure layer that bridges traditional software and AI-native systems.

As AI agents become operational entities, applications will require:

- semantic interfaces
- safe AI interaction layers
- workflow-aware abstractions
- permission-aware execution

MCPify provides the compiler that transforms software into AI-operable environments.

Final Positioning

MCPify is not:

- a wrapper generator
- an API tool
- a simple MCP scaffold

MCPify is:

“An AI Enablement Compiler”

A system that converts applications into structured, secure, AI-operable environments automatically.

Taglines

- “Compile software into AI-operable systems.”
- “Turn applications into AI-native environments.”
- “The AI interface layer for software.”
- “Make any application usable by AI agents.”
- “From software to AI-operable systems instantly.”